

Testy aplikacji

Projekt: Detektor Nieoznakowanych
Wersja aplikacji: 0.1.0-beta
Data wykonania: 2026-06-03

Środowisko testowe

Element	Wartość
System projektu	Windows, PowerShell
Android Gradle Plugin	8.5.1
Gradle Wrapper	8.7
Kotlin	1.9.24
<code>compileSdk / targetSdk</code>	35 / 35
<code>minSdk</code>	26
Telefon testowy	Samsung SM-S901B
Android telefonu	16, API 36
Pakiet aplikacji	<code>com.example.unmarkeddetector</code>

Komendy wykonane podczas testów

```
.\gradlew.bat testDebugUnitTest assembleDebug lintDebug bundleRelease --console=plain

$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adb install -r app/build/outputs/apk/debug/app-debug.apk
& $adb shell dumpsys package com.example.unmarkeddetector
& $adb shell cmd appops get com.example.unmarkeddetector RUN_ANY_IN_BACKGROUND
```

Podsumowanie wyników

Obszar	Wynik
Testy jednostkowe	15 testów, 0 błędów, 0 porażek, 0 pominiętych
Build debug APK	Sukces
Lint debug	0 błędów, 77 ostrzeżeń
Build release AAB	Sukces
Instalacja APK na telefonie	Sukces
Uprawnienie pracy w tle na telefonie	RUN_ANY_IN_BACKGROUND: allow

Scenariusze testowe

ID	Scenariusz	Kroki	Oczekiwany wynik	Wynik rzeczywisty
T-01	Walidacja poprawnej tablicy po OCR	Uruchomić testy <code>PlateValidatorTest</code> .	Poprawne ciągi tablic są normalizowane do kanonicznej postaci.	Testy przeszły. Przykład <code>P025Y7P</code> .
T-02	Odrzucanie tekstów niebędących tablicami	Uruchomić testy <code>PlateValidatorTest</code> i <code>DetectPlateUseCaseTest</code> .	Zwykłe słowa i za krótkie formaty nie są uznawane za tablice.	Testy przeszły. Przykład <code>POLSKIE</code> , <code>GENERATOR</code>
T-03	Korekta typowych błędów OCR	Uruchomić test <code>repairs likely OCR prefix mistakes</code> .	Typowe pomyłki, np. <code>OOT F038</code> , są naprawiane do poprawnego prefiksu.	Test przeszedł: wynik
T-04	Filtrowanie kandydatów po	Uruchomić testy <code>DetectPlateUseCaseTest</code> .	Kandydaci poniżej progu	Test przeszedł. Kandydat <code>0.79</code> jest odrzucany.

ID	Scenariusz	Kroki	Oczekiwany wynik	Wynik rzeczywisty
	pewności OCR		pewności są odrzucani.	
T-05	Liczenie unikalnych tablic w sesji	Uruchomić test <code>counts only unique plates in session</code> .	Powtórzenia tej samej tablicy nie zwiększają licznika.	Test przeszedł. Dwie n licznik <code>2</code> , powtórki ni
T-06	Cooldown alertu dla tej samej tablicy	Uruchomić test <code>applies alert cooldown for same plate</code> .	Ten sam rekord z bazy nie generuje natychmiast wielu alertów.	Test przeszedł. Drugi z tablicy został zabloko
T-07	Stabilizacja odczytu w czasie	Uruchomić testy <code>PlateTrackManagerTest</code> .	Stabilna detekcja pojawia się dopiero po powtórzonym potwierdzeniu.	Testy przeszły. Pojedny emituje stabilnej tablic
T-08	Wygenerowanie zdarzenia alertu	Uruchomić test <code>TriggerAlertUseCaseTest</code> .	Dopasowana tablica tworzy <code>AlertEvent</code> i wywołuje dispatcher.	Test przeszedł. Zdarze rekord i pewność.
T-09	Kompilacja aplikacji debug	Uruchomić <code>assembleDebug</code> .	Powstaje APK debug.	APK powstał w <code>app/build/outputs/assembleDebug.apk</code> .
T-10	Analiza lint	Uruchomić <code>lintDebug</code> .	Brak błędów blokujących.	Lint zakończył się wyn <code>warnings</code> .
T-11	Przygotowanie paczki release	Uruchomić <code>bundleRelease</code> .	Powstaje Android App Bundle.	AAB powstał w <code>app/build/outputs/bundle/release.aab</code> .

ID	Scenariusz	Kroki	Oczekiwany wynik	Wynik rzeczywisty
T-12	Instalacja APK na telefonie	Wykonać <code>adb install -r app-debug.apk</code> .	Instalacja kończy się sukcesem.	<code>Performing Streamed Install</code> <code>Success</code> .
T-13	Sprawdzenie manifestu na telefonie	Wykonać <code>adb shell dumpsys package com.example.unmarkeddetector</code> .	Pakiet ma właściwą wersję, target SDK i uprawnienia.	<code>versionName=0.1.0-l</code> widoczne uprawnienia powiadomień i foreground
T-14	Sprawdzenie pracy w tle	Wykonać <code>adb shell cmd appops get ... RUN_ANY_IN_BACKGROUND</code> .	System nie blokuje pracy w tle.	Wynik: <code>RUN_ANY_IN_B</code>
T-15	Skanowanie po przejściu aplikacji w tło	Uruchomić skanowanie, przejść do innej aplikacji i obserwować dalszą pracę foreground service.	Skanowanie działa mimo niewidocznego preview.	Test manualny na tele potwierdzony 2026-01 w tle.

Zakres testów jednostkowych

Plik testowy	Liczba testów	Sprawdzany obszar
<code>PlateValidatorTest.kt</code>	6	Format tablic, normalizacja OCR, odrzucanie błędnych tekstów.
<code>PlateTrackManagerTest.kt</code>	3	Stabilizacja wyników OCR w czasie i potwierdzanie detekcji.
<code>PlateSessionManagerTest.kt</code>	2	Unikalne tablice w sesji i cooldown alertów.
<code>DetectPlateUseCaseTest.kt</code>	3	Filtrowanie kandydatów OCR i deduplikacja wyników.
<code>TriggerAlertUseCaseTest.kt</code>	1	Tworzenie i wysyłanie zdarzenia alertu.

Wnioski z testów

Aplikacja spełnia podstawowe kryteria wersji beta: kompiluje się, przechodzi testy jednostkowe, generuje APK i AAB oraz instaluje się na fizycznym telefonie. Najważniejsza logika domenowa jest pokryta testami automatycznymi. Test ręczny potwierdził również działanie skanowania po przejściu aplikacji w tło.

Ostrzeżenia lint nie blokują uruchomienia aplikacji. Dotyczą głównie zależności, drobnych optymalizacji layoutów i rekomendacji technicznych. Nie wykryto błędów krytycznych.

Testy rekomendowane przed finalną prezentacją

1. Test drogowy w dzień na kilku rzeczywistych tablicach.
2. Test drogowy wieczorem lub przy słabszym świetle.
3. Test odmowy uprawnień na czystej instalacji aplikacji.
4. Test działania overlay nad aplikacją nawigacji.
5. Test termiczny po minimum 20 minutach ciągłego skanowania.